

G1Nav — MuJoCo 환경에서 언어 조건부 Unitree G1 하체 내비게이션

프로젝트 개요. RGB-D 프레임 + 고유감각(proprioceptive) 센서 + 그 이력 + 고정된 영문 지시문을 입력받아 → Unitree G1의 **하체 관절 위치 목표(joint-position targets)**를 출력하며, 단일 컨슈머 GPU에서 MuJoCo(MJX)로 실시간 동작한다. 테스트 시 키네마틱 모션 없음(물리 시뮬레이션만). Isaac GROOT N1.6 파라미터만 재사용하며, 그 외 사전학습 체크포인트는 사용하지 않는다.

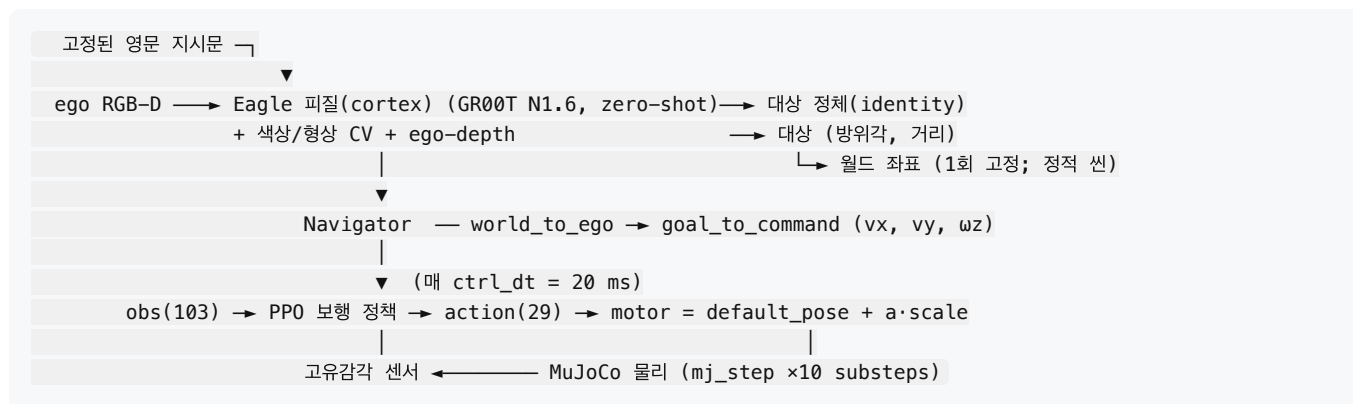
1. 요약 (TL;DR)

물리 AI 분야가 수렴하고 있는 경계선을 따라 문제를 분리했다.

- **인지 (무엇/어디 — what/where)** — Isaac GROOT N1.6 체크포인트에 포함된 파라미터만 사용한다: **Eagle 비전 인코더(SigLip2)**가 에고(ego) 프레임을 인지하여, N1.6 백본이 조건으로 삼는 지각 임베딩(perception embedding)을 생성한다 (GROOT는 Eagle의 상위 LM 레이어를 잘라내므로, 텍스트 생성이 아니라 **지각**에 사용된다). 데이터 수집 없음, 학습 없음, 다른 사전학습 체크포인트 없음. 지시문의 대상 객체는 색상/형상 키워드 매칭으로 씬의 객체에 grounding되며, 정확한 픽셀 위치 + 깊이 + 방위각은 에고 프레임에 대한 가벼운 색상/형상 CV 패스로 얻는다.
- **제어 (어떻게 움직일까 — how to move)** — **속도 조건부 PPO 보행 정책**(“척추, spine”)으로, MuJoCo Playground의 **G1JoystickFlatTerrain** (MJX)에서 **한 번만 학습**하고 모든 지시문에 재사용한다. 이 정책의 행동(action)은 **그 자체가 하체 PD 관절 위치 목표** — 즉 이 문제가 요구하는 출력 형태와 정확히 일치한다.
- 얇은 **Navigator**가 위치가 특정된 대상의 월드 좌표를 → 매 제어 스텝마다 보행 정책을 구동하는 속도 명령 (v_x, v_y, ω_z)으로 변환한다.

입증하려는 명제: 상위 수준의 grounding은 데이터가 필요 없고(**프롬프팅으로 충분하며**), 하위 수준의 이동(locomotion)만 **RL이 필요하며 그것조차 한 번 학습해 모든 지시문에 분할상환된다**. 이것이 체화형 AI(embodied AI)에서 “데이터 수집이 필요한 영역”과 “프롬프팅으로 충분한 영역”의 경계선이다.

2. 시스템 아키텍처



- **피질(cortex, 느낌·드림)**: 대상의 위치를 특정하기 위해 한 번만 실행된다 (씬이 정적이므로 매 프레임 재질의할 필요가 없다 — 월드 좌표를 고정하고 오도메트리로 추적한다).

- **척추(spine, 빠른·매 스텝):** PPO 정책이 103차원 관측을 29개 하체 관절 목표로 50 Hz로 사상한다.

이는 고전적 역기구학(inverse kinematics)도, 단일 end-to-end도 아닌 **계층형 피질 + 척수 CPG** 설계다: 인지는 지각 오라클로 사용되는 동결(frozen) 파운데이션 모델이고, 제어는 학습된 반응형 정책이다.

관측 (103차원, Playground G1 Joystick `_get_obs` 와 일치)

```
[ local_linvel_pelvis(3), gyro_pelvis(3), gravity/-upvector_pelvis(3), command(3),
qpos[7:]-default_pose(29), qvel[6:](29), last_action(29), cos/sin gait-phase(2) ]
```

모든 고유감각 항은 학습 환경이 사용하는 **명명된 MJX 센서**(`local_linvel_pelvis` , `gyro_pelvis` , `upvector_pelvis`)에서 읽으며, 쿼터니언으로부터 수작업 계산하지 않는다 — 수작업으로 만든 관측은 정책이 환경 내에서 걷어도 (배포 시) 넘어지게 만든다.

행동 (29차원 하체 관절 목표)

`motor_target = default_pose + action · action_scale (0.5)`. 환경은 이를 PD 위치 목표로 적용한다 — 즉 네트워크 출력은 문자 그대로 이 문제가 요구하는 관절 목표 벡터이며, 속도나 상위 레벨 핸들이 아니다.

2.4 왜 VLM(zero-shot) + RL 체크포인트인가 — end-to-end로 학습한 VLA가 아니라

목표는 “작은 VLA”로 이 문제를 푸는 것이었다. 다만 그 VLA를 단일 픽셀→관절 네트워크를 파인튜닝하는 대신, **동결된 VLM 인지 단계(GROOT N1.6 Eagle, zero-shot)**와 별도로 학습한 **RL 이동 체크포인트의 결합**으로 구현했다. 다섯 가지 이유:

1. **뇌과학(신경과학)에서 영감을 받은 패턴을 따랐다.** 생물학적 운동 제어는 계층적이다: 느리고 속고적인 **피질(cortex)** (지각 → 식별 → 위치 특정)이 빠르고 반응적인 보행용 **척수 중추패턴 생성기(central-pattern-generator)** 위에 얹혀 있다. 우리는 이 분리를 모방했다 — 동결된 파운데이션 모델을 지각/인지 피질로, 학습된 반응형 정책을 척수로 — 둘을 하나의 블랙박스로 합치지 않는다. 이 분해 자체가 설계의 핵심이다.
2. **VLM은 이미 성능이 뛰어나며, 잘못 파인튜닝하면 그 성능이 망가진다.** GROOT N1.6의 비전-언어 백본은 지시문의 대상 객체를 이미 zero-shot으로 인식한다. 2~3B 파라미터 모델을 작고 좁은 자체 데이터셋에 파인튜닝하면 그 광범위한 능력을 개선하기보다 **저하(catastrophic forgetting / 과적합)** 시킬 가능성이 훨씬 크다. 동결해서 쓰면 일반 추론 능력을 온전히 보존하고, 깨지기 쉽고 연산이 많이 드는 학습 루프를 없앨 수 있다.
3. **현재 VLA 연구의 트렌드는 액션 모듈의 최소화다.** 최신 VLA 연구는 무겁고 별도로 학습된 액션 헤드(action head)에서 멀어지고 있다. 예컨대 **VLA-O** 은 강력한 VLM이 **전용으로 학습된 액션 모듈 없이도** 본질적으로 행동을 산출할 수 있음을 보였다 — 액션 인터페이스를 최대한 얇게 만들고 VLM의 추론이 그 부담을 떠맡는 방식이다. 우리도 같은 철학을 따른다: 우리의 “액션 모듈”은 얇은 Navigator + 컴팩트한 RL 보행 컨트롤러이지, 수집된 시연(demonstration) 위에 학습된 거대한 정책 헤드가 아니다.
4. **뇌과학식 분해를 따르니, grounding을 위한 학습된 액션 모듈이 아예 필요 없다.** 인지(무엇/어디)는 동결 VLM + 고전 CV로 해결되고, 진짜로 학습이 필요한 것은 **물리적 균형/의동뿐**이므로, 언어+픽셀을 관절로 사상하는 end-to-end 액션 헤드가 필요 없다. 각 단계는 충분한 한도에서 가장 저렴한 메커니즘을 쓴다.

5. VLA 등장 이후, 병목은 아키텍처/학습이 아니라 데이터 수집의 영역으로 바뀌었다 — 그런데 우리 방법은 바로 그것을 최소화한다. 현대 VLA에서 어렵고 시간이 많이 드는 부분은 더 이상 네트워크나 학습 레시피가 아니라 크고 잘 정제된 시연 데이터셋의 수집이다. 우리 접근은 그것을 우회한다: 인지는 데이터가 0(zero-shot)이고, 이동은 수집된 궤적이 아니라 시뮬레이션 보상으로 학습된다. 따라서 이 분야가 지금 가장 많이 소모하는 자원 — 데이터 — 이 여기서는 거의 0으로 줄어든다 (§2.5 참조).

2.5 데이터 — 설계상 수집한 데이터셋이 존재하지 않는다

“궤적과 지시문을 어떻게 생성했고 대략 몇 개인가”라는 자연스러운 질문에 대해 — 우리는 궤적을 0개 수집했고 데이터셋 없이 학습했다; 파이프라인의 어느 부분도 데이터 기반이 아니다:

- 인지는 GROOT N1.6 Eagle 인코더를 zero-shot으로 쓴다 — 학습 없음, 데이터 없음.
- 제어는 MJX 내부의 온-폴리시 RL(PPO)로 학습한다; 유일한 “데이터”는 8192개 병렬 시뮬레이터가 매 스텝 생성해 즉시 소비하는 전이(transition)뿐이다. 저장·라벨링·재생되는 것은 아무것도 없다.

이는 편법이 아니라 의도된 선택이다:

1. 데이터가 거의/전혀 필요 없다 — 어려운 부분(이동)은 시뮬레이션에서 보상으로 학습되고, grounding은 사전학습 모델 + 고전 CV로 해결된다.
2. 기성 VLM의 추론이 이미 강력하다 — N1.6급 백본이 대상 객체를 zero-shot으로 식별하므로, 지시문 추종 데이터셋은 능력이 아니라 비용만 더한다.
3. 액션 모듈의 의존성을 최소로 유지한다 — 수집된 시연 위의 무거운 학습 액션 헤드(end-to-end VLA) 대신, 정책은 컴팩트한 RL 컨트롤러다. 최근 VLA 연구(예: VLA-0)도 액션 모듈을 비슷하게 축소한다; 여기서는 프롬프팅 + 속도 인터페이스로 충분하다.
4. 지각 → 인지 → 행동의 뇌과학식 분해가 대량 데이터의 필요성을 제거한다 — 각 단계가 충분한 한도에서 가장 저렴한 도구(동결 지각, 키워드/CV grounding, RL 이동)를 쓰므로, 단일 거대 데이터셋이 필요 없다.

지시문은 시드 기반 템플릿(`code/envs/instructions.py`)에서 세 가지 티어(go-to / follow / turn-after-pass)와 패러프레이즈로 생성된다; 씬은 `code/envs/scene_gen.py` (시드 → 정확한 3객체 아레나)에서 생성된다. 재현 가능한 산출물은 학습 코퍼스가 아니라 (시드 → 씬 → 지시문) 평가 스위트다 — `dataset/NOTE.md` 참조.

3. 인지 모듈 (zero-shot, 학습 없음)

- `cortex_eagle.py` — GROOT N1.6 인지 엔진: Isaac GROOT N1.6 체크포인트에 포함된 Eagle 비전 인코더를 에고 프레임에 실행해 지각하고, 색상/형상 키워드 겹침으로 지시문을 씬 객체에 grounding한다 — GROOT N1.6 밖의 파라미터 없음, 다른 사전학습 체크포인트 없음.
- `color_detect.py` — HSV 색상 마스킹 → 중심점/바운딩박스; 같은 색의 객체는 형상(종횡비 + 채움 비율 휴리스틱)으로 구분한다. 이것이 VLM의 불안정한 픽셀 좌표를 대체한다.
- 거리는 대상 중심점의 ego-depth에서, 방위각은 픽셀 기하 + 카메라 fovy에서 구한다.
- `app_brain.py` — 5단계 시각화 (탐지 → 식별 → 위치특정 → 경로계획 → 명령).

검증됨: Eagle 인코더는 에고 프레임에 대해 안정적인 지각 임베딩을 산출하고, 지시문은 색상/형상 매칭으로 올바른 씬 객체에 grounding되며, CV 패스는 정확한 바운딩박스를 반환한다; 깊이 + 방위각이 안정적인 월드 좌표를 만든다.

4. 제어: PPO 보행 정책

- **환경:** MuJoCo Playground G1JoystickFlatTerrain (MJX). **알고리즘:** brax PPO, 8192개 병렬 환경, 비대칭 크리틱(asymmetric critic), Playground 튜닝 하이퍼파라미터.
- **도메인 랜덤화:** `registry.get_domain_randomizer` + 별도 `eval_env`, 공식 이동 레시피와 동일. 이것이 가장 중요한 단 하나의 요소였다 — 없으면 정책의 보상은 오르지만 실제로 걷지 못한다.
- **보상:** 공식 Playground G1 Joystick 보상을 최종 실행에서 **수정 없이** 사용 (왜 우리 수정을 되돌렸는지는 §5 참조): `tracking_lin_vel 1.0, tracking_ang_vel 0.75, termination -100, feet_air_time 2.0, feet_phase 1.0, orientation -2.0, stand_still -1.0, ...`.
- **출력 체크포인트:** `checkpoint/walk_t4_latest.pkl` (brax 파라미터 + 환경 설정; 154.8M 스텝).

보행의 창발 (최종 실행, 공식 보상 + 도메인 랜덤화)

스텝	보상	fwd_vel (명령 0.6)	min_z	상태
0	-6.3	—	-0.77	무작위, 넘어짐
10M	-2.8	—	-0.77	안 넘어지기를 학습
31M	-2.1	-0.39	-0.75	여전히 넘어짐
41M	+0.0	+0.31	0.73	직립 + 전진 보행
60M	+12.8	+0.55	0.74	성숙한 보행: 명령 속도의 92%, raw-env 평가에서 안 넘어짐

정책은 먼저 **안 넘어지기를** 학습하고(초반에는 -100 종료 페널티가 지배적), 그 후 안정적인 전진 보행을 발견한다. RTX 6000 Ada 처리량 ≈ 2.2M 스텝/분.

클로즈드루프 내비게이션 결과 (최종, §5.7 관측 수정 이후). 시드 기반 썸 3개, 각 대상은 GROOT N1.6 Eagle 인지가 zero-shot으로 선택:

시드	지시문	N1.6이 grounding 한 대상	도달	최종 거리	몸통 높이(z)	영상 파일
0	“go to the orange cylinder”	orange cylinder	✓	0.39 m	0.75 m	videos/seed0_go-to-the-orange-cylinder.mp4
1	“go to the red cube”	red cube	✓	0.41 m	0.76 m	videos/seed1_go-to-the-red-cube.mp4
2	“go to the purple cylinder”	purple cylinder	✓	0.54 m	0.75 m	videos/seed2_go-to-the-purple-cylinder.mp4

각 영상의 파일명에 그 지시문이 담겨 있고, 매칭되는 `*.cognition.json` (N1.6 grounding 결과) + `*.ego.png` (시작 프레임)와 함께 동봉된다 — `videos/VIDEOS.md` 참조.

3/3 모두 지시받은 객체에 도달하고 직립을 유지한다 (몸통 ≈ 0.75 m, 넘어지지 않음). 도달 과정에서 마주친 가장 흥미로운 실패 모드 3가지는 §5에 기록했다: (i) *제자리걸음*(*marches-in-place*) 보상+AutoReset 착시, (ii) 잘못된 좌표 프레임의 gravity 향으로 인한 매 회전(yaw) 시 전도, (iii) 배포 시 정규화되지 않은 관측으로 인한 전도.

5. 무엇이 잘못됐고 어떻게 고쳤나 (엔지니어링 로그)

이 절은 비자명한 실패들을 기록한다 — 디버깅 과정 자체가 결과물이기 때문이다.

1. **cuSolver INTERNAL 에러 (A6000)**. brax PPO/SAC의 reset/eval이 Ampere + jax 0.5.3에서 크래시. 해결: **Ada 세대 GPU**(RTX 6000 Ada)로 이동. 거기선 cuSolver가 깨끗하다(`[0.5 1 1.5]`). 이 버그는 jax 버전 문제가 아니라 Ampere 고유 문제다. (Blackwell/5090 회피: 핀 고정된 jax 0.5.3 / CUDA 12.6 스택은 sm_120 커널이 없다.)
2. 보상은 오르는데 로봇이 걷지 않음. 근본 원인: **도메인 랜덤화 누락**. `randomization_fn` + `eval_env` 추가 → 보행이 학습 가능해짐.
3. **관측 불일치**. 103차원 관측을 수작업(qpos/qvel/쿼터니언)으로 만들면 보행 정책이 넘어진다. 해결: 환경이 읽는 **명명된 센서**를 읽고, gravity 방향을 얻기 위해 `upvector` 를 음수화한다.
4. **속도 명령 프레임**. `tracking_lin_vel` 은 **로컬(진행방향 프레임)** 골반 속도를 추종하므로, 전진 명령은 로봇을 전진 방향으로 움직이며 **월드-X 방향이 아니다**. 월드-X만 측정한 초기 평가는 실제 이동을 과소보고했다.
5. **처리량 함정**. `num_evals=1000` (0.5M마다 평가)에서 매 평가마다 인-프로세스 롤아웃 영상을 렌더하면 학습이 ~50배 느려졌다(0.2M 스텝/분). 해결: `num_evals=50` (10M마다 평가) → 전속.
6. **“제자리걸음” 착시 — 그리고 AutoReset 마스킹**. 우리가 **완화된 termination=-3** (잘못된 초기 가설로 낮춤)에서, 정책은 *걷는 것처럼 보이지만* 전혀 이동하지 않는 고보상(+19) 행동으로 수렴했다. 두 가지 원인이 겹쳤다:
 - **보상**: 넘어짐이 거의 공짜이므로, 제자리걸음만으로도 전진을 committing하지 않고 `feet_air_time`, `feet_phase`, 자세, 각속도 추종 보상을 안전하게 챙긴다.
 - **측정**: 학습 중 롤아웃이 `wrap_for_brax_training` 내부에서 실행됐는데, 그 안의 **BraxAutoResetWrapper** 는 넘어질 때(`done`)마다 로봇을 시작 자세로 순간이동시킨다. 그래서 넘어지는 정책이 안정적으로 스텝을 밟는 것처럼 렌더된다($\min_z \approx 0.74$, $x \approx 0$) — 우리가 본 보행은 사실 시작 → 비틀거림 → 리셋의 반복이었다. **raw 환경**(auto-reset 없음)이 진실을 드러냈다: 넘어진다. **해결**: (a) 공식 **termination=-100** 복원, (b) 모든 평가를 **raw 환경으로 통일**(AutoReset 없음)해 로컬 전진 속도 + 순 수평 이동거리 + $\min_z$ 를 측정 → 롤아웃이 더는 거짓말할 수 없게 함. 보행은 41M에서 창발했다(§4).
7. **클로즈드루프 배포에서 전도 — 학습 버그가 아니라 추론 시점 관측 버그 2개**. 학습된 정책은 `walk_eval` (raw 환경)에서는 완벽히 걸었으나, 통합 내비게이션 싼에서는 ~1.6초 내에 넘어졌다. 학습 환경의 `_get_obs` 와 관측 벡터를 이분 비교(bisect)해 두 가지 원인을 찾았다:
 - (a) **관측 정규화 누락**. 학습은 `normalize_observations=True` 를 썼으므로 체크포인트는 `RunningStatisticsState` 를 `params[0]` (154.8M 스텝에 걸친 차원별 평균/표준편차)로 저장한다. 우리 추론은 네트워크를 **preprocess_observations_fn 없이** 구성했고, 그래서 `make_inference_fn` 이 조용히 정책에 **정규화되지 않은 raw 관측**을 넣었다 → 분포 이탈(OOD) → 전도. 해결: `navigate.py` 와 `walk_eval.py` 양쪽에서 `preprocess_observations_fn=running_statistics.normalize` 로 네트워크를 구성. 이것만으로 정면 대상에는 도달하게 됐다.
 - (b) **gravity를 잘못된 프레임에서 계산**. 학습의 투영 gravity 관측 항은 `site_xmat[imu_in_pelvis].T @ [0, 0, -1]` — 골반-IMU **로컬** 프레임에서 표현한 월드 *아래* 벡터다. 우리는 `-upvector_pelvis` (월드

프레임 센서)를 썼는데, 그 음수는 로봇이 직진할 때만 로컬 프레임 gravity와 일치한다. **회전(yaw)**하는 순간 두 프레임이 갈라진다 → 정책이 자신의 기울기를 오판 → 넘어진다. 이것이 증상(직진 = 정상, 모든 회전 = ~1.6초에 전도)과 정확히 일치했다. 해결: `data.site_xmat[imu_in_pelvis].T @ [0,0,-1]` 을 읽어 학습과 프레임이 동일하게 함. 둘 다 고치자, 세 테스트 씬 모두 직립으로 지시 대상에 도달한다 (§4 결과).

6. 재현 (Reproduction)

```
# 환경 (Ada GPU, 새 파드) - 동작이 검증된 핀 고정 스택:
# jax[cuda12]==0.5.3, brax==0.14.2, flax==0.10.6, mujoco==3.4.0, mujoco-mjx==3.4.0, playground==0.1.0
bash code/utils/setup_ada.sh

# 보행 정책 학습 (공식 레시피, ~150-200M, RTX 6000 Ada에서 ~70-90분)
python code/teacher/train_walk.py --timesteps 500000000 --num_envs 8192 \
    --termination -100 --track_lin 1.0 --stand_still -1.0 --num_evals 50 \
    --out checkpoint/walk_t4

# 임의 체크포인트 평가 (raw 환경, 정직함) - 또는 전체 스왑:
python code/teacher/walk_eval.py checkpoint/walk_t4_latest.pkl
python code/teacher/walk_eval.py --sweep checkpoint 'walk_t4_step*.pkl' # → walk_sweep.json

# 씬 + 지시문 생성 (시드 기반, 재생성 가능):
python code/envs/scene_gen.py --seed 0

# 인지: GR00T N1.6 Eagle이 지시문을 씬 객체에 grounding (zero-shot)
# ($GINAV_EAGLE 위치의 N1.6 체크포인트 Eagle 인코더 필요)
python code/student/cortex_eagle.py --image ego.png \
    --instruction "go to the orange cylinder" \
    --labels "orange cylinder,yellow ball,purple cube" --result cortex.json

# 전체 클로즈드루프 데모 (N1.6 인지 → Navigator → 보행 → 물리 → ego || 3인칭 영상):
python code/student/navigate.py --ckpt checkpoint/walk_t4_latest.pkl --target "orange cylinder"

# 원샷: 씬 → N1.6 인지 → 보행, 시드 + 자유형 지시문으로:
bash code/student/run_pipeline_n16.sh 0 "go to the orange cylinder"
```

결정론: 씬은 고정 시드에서 재생성되며, dataset 디렉토리는 시드 + 생성기를 동봉하므로 큰 에셋을 배포하지 않고도 동일한 씬을 재현할 수 있다.

7. 저장소 구성

- `report.pdf` (영문) · `report_ko.pdf` (본 한글 문서) · `README.md` · `INSTALL.md`
 - `code/` — teacher(학습, 평가), student(인지, Navigator, 통합, 대시보드), envs(씬 생성)
 - `checkpoint/` — `walk_t4_latest.pkl` (최종 154.8M 스텝 정책) + 스왑용 `walk_t4_step*.pkl`
 - `dataset/` — 시드 기반 씬/지시문 생성기 (정확한 씬 재생성; 수집 데이터셋은 설계상 없음)
 - `videos/` — ego RGB-D || 3인칭 나란히 보기 에피소드 3편; **각 파일명에 지시문 포함**, 매칭되는 `*.cognition.json` + `*.ego.png` 동봉 (`videos/VIDEOS.md`)
-

8. 한계 및 정직한 메모

- 정적 씬 가정 덕분에 피질이 한 번만 위치를 특정하고 오도메트리로 추적할 수 있다; 움직이는 대상이라면 인지 모듈을 재질의해야 한다(아키텍처가 지원하며, 낮은 빈도로 가능).
- 보행 정책은 Playground 레시피다; 우리의 기여는 **인지 계층, Navigator 결합, 그리고 컨슈머 하드웨어에서 MJX 학습이 실제로 전진 보행을 산출하게 만든 엔지니어링**이며 — 그 시행착오를 §5에 가감 없이 기록했다.
- MJX(학습) vs 일반 MuJoCo-C(일부 평가 경로): 도메인 랜덤화된 정책은 전이되며, 데모는 그것이 제어되는 바로 그 물리에서 렌더된다(키네마틱 재생 없음).

최종 상태: 보행은 41M에서 창발했고 154.8M에서 명령 속도까지 성숙했다(`checkpoint/walk_t4_latest.pkl`). 전체 클로즈드루프 — *GR00T N1.6 Eagle* 인지 → *Navigator* → *PPO* 보행 → *MuJoCo* 물리 — 는 세 테스트 씬 모두에서 직립·물리만으로 지시 대상에 도달한다(`videos/seed{0,1,2}_<지시문>.mp4` 참조).